第10章: 用語集 — わからない言葉が出てきたら、まずここ

> この章は **辞書（じしょ）** です。最初から最後まで通して読む必要はありません。本資料の他の章（や実際のコード、エラーメッセージ）を読んでいて「**この言葉、なんだっけ？**」と思ったら、ここに戻って引いてください。
>
> 各用語には **読み仮名（よみがな）・やさしい意味・たとえ話** を付けました。プログラミングの専門用語は、英語の略語だらけで最初はとっつきにくいものですが、ほとんどは「身近な何かにたとえる」と一気に腑に落ちます。たとえ話は厳密さより「最初のイメージのつかみやすさ」を優先しています（だから細部は実際と違う場合があります。それでよいのです。まず手がかりを得るのが目的です）。
>
> 用語は **分野ごと** にまとめてあります。下の「目次」から、探したい分野へ飛んでください。分野をまたいで同じ言葉が複数の場所に出ることもありますが、それは「その分野ではこういう意味で使う」を示すためです。

この章で学ぶこと

- **アプリ全体に関わる言葉** — フロントエンド／バックエンド／API／クライアント／サーバー／リクエストとレスポンス 等
- **開発の道具に関わる言葉** — エディタ／ターミナル／npm／pip／ビルド／パッケージ 等
- **Git（バージョン管理）の言葉** — コミット／ブランチ／プッシュ／プル／マージ／コンフリクト 等
- **HTML・CSS の言葉** — DOM／レンダリング／ボックスモデル／Flexbox／擬似要素／CSS変数／clamp／box-sizing 等
- **JavaScript・TypeScript の言葉** — 変数／関数／非同期／Promise／型／async/await 等
- **React の言葉** — コンポーネント／props／state／useState／useEffect／JSX／SPA 等
- **Python・Django の言葉** — モデル／QuerySet／ORM／マイグレーション／シリアライザ／セッション／CSRF／CORS／デコレータ／シグナル／ページネーション／WSGI／Gunicorn 等
- **このアプリ特有の言葉** — 工数（こうすう）／直（ちょく）／288枠／休憩文字列／ショップ／工数区分定義 等
- **運用・Azure の言葉** — デプロイ／フレキシブルサーバー／環境変数／staticfiles 等
- **テストの言葉** — モック／アサーション／Vitest 等
- **困ったときの調べ方**

目次

1. この用語集の使い方
2. アプリ全体に関わる言葉
3. 開発の道具に関わる言葉
4. Git（バージョン管理）の言葉
5. HTML・CSS の言葉

6. JavaScript・TypeScript の言葉
7. React の言葉
8. Python・Django の言葉
9. このアプリ特有の言葉
10. テストに関わる言葉
11. 運用・Azure の言葉
12. 記号・略語のはやわかり表
13. 困ったときの調べ方
14. この章のまとめ

---

1. この用語集の使い方

1.1 読み方の3パターン

この章は、次の3通りの使い方を想定しています。

1. **逆引き（ぎゃくびき）** — 「あの言葉なんだっけ？」と思ったら、目次から分野を選んで探す。これが基本の使い方です。
2. **分野まとめ読み** — たとえば「Reactの章を読む前に、Reactの言葉だけ先に眺めておく」。section 7 だけ読む、という使い方。
3. **ブラウザの検索** — このファイルを開いて `Ctrl + F`（ブラウザやエディタの「ページ内検索」）で、探したい言葉をそのまま打ち込む。これが一番速いことも多いです。

> **▼ こう使えば成功:** 他の章で「QuerySet」という言葉が出てきて意味がわからない → この章を開く → `Ctrl + F` で「QuerySet」と打つ → 説明にジャンプ →「ああ、データベースから取り出した行の集まりのことか」と理解 → 元の章に戻る。この往復ができれば、難しい技術文書も少しずつ読めるようになります。

1.2 用語の書き方の約束

この章では、用語を次の形で説明します。

> **用語名（よみがな：ひとことの意味）** やさしい説明。**たとえ:** 身近なものでのたとえ話。

「よみがな」は、英語の場合はカタカナ読みを、日本語の場合はひらがな読みを書きます。たとえば「API（エーピーアイ）」「工数（こうすう）」のように。

> ⚠️ **たとえ話は「入口」です。**たとえ話はあくまで最初のイメージをつかむためのもので、技術的に100%正確とは限りません。慣れてきたら、たとえ話を卒業して「本当の意味」で理解していってください。たとえ話の役目は「最初のとっつきにくさを消す」ことです。

---

2. アプリ全体に関わる言葉

このアプリ「業務工数システム」がどういう作りなのか、その全体像に関わる言葉です。まずここを押さえると、他の章が一気に読みやすくなります。

アプリケーション（あぷりけーしょん：目的を持ったソフトウェアのまとまり） 略して「アプリ」。何かの仕事をするために作られたプログラムのまとまり。このアプリは「従業員の働いた時間（工数）を記録・集計する」ためのソフトウェアです。**たとえ:** 家電でいえば「電子レンジ」。温めるという目的のために、ボタン・ヒーター・タイマーが一式そろっている。

Webアプリ（うぇぶあぷり：ブラウザで使うアプリ） インストールせず、ブラウザ（Chrome、Edge など）で開いて使うアプリ。このアプリもWebアプリで、URLを開けば使えます。**たとえ:** 自宅にDVDプレイヤー（インストール型）を置くのではなく、テレビ局の放送（ブラウザで受信）を見るイメージ。手元に本体を持たなくていい。

フロントエンド（ふろんとえんど：画面側。利用者が直接さわる部分） 利用者の目に見える「画面」と、その操作を担う部分。このアプリでは **React（リアクト）** で作られた、`frontend/` フォルダの中身がそれです。ボタン・表・入力欄など。**たとえ:** レストランの「ホール（客席・接客係）」。お客さんが直接やりとりする場所。

バックエンド（ばっくえんど：裏側。データを保管し計算する部分） 画面の裏で、データを保存したり、計算したり、ルールを守らせたりする部分。このアプリでは **Django（ジャンゴ）** で作られた、ルート直下（ `kosu/` や `hozen_another/` ）の中身がそれです。**たとえ:** レストランの「厨房（ちゅうぼう）」。お客さんからは見えないが、注文を受けて料理（データ処理）をする。

サーバー（さーばー：要求に応えて何かを返すコンピュータ／プログラム） 「これをくれ」という要求（リクエスト）を受け取って、結果を返すコンピュータやプログラム。このアプリのバックエンドはサーバーとして動きます。**たとえ:** 食堂の調理場。「カレーください」と言われたら、カレーを作って渡す係。

クライアント（くらいあんと：サーバーに要求を出す側） サーバーに「これがほしい」と要求する側。ブラウザ（＝フロントエンド）がクライアントです。**たとえ:** 食堂のお客さん。「カレーください」と注文する側。

リクエスト（りくえすと：要求。クライアント→サーバーへの「これをくれ」） クライアントがサーバーへ送る「お願い」。「工数データをください」「このデータを保存してください」など。**たとえ:** 注文票。「カレー1つ」と書いて厨房に渡す紙。

レスポンス（れすぽんす：応答。サーバー→クライアントへの「はいどうぞ」） サーバーがリクエストに対して返す結果。データそのものや、「成功しました」「失敗しました」という知らせ。**たとえ:** 厨房から出てきた料理、または「品切れです」という返事。

API（エーピーアイ：プログラム同士がやりとりするための窓口） Application Programming Interface（アプリケーション・プログラミング・インターフェース）の略。「こういう要求を送れば、こういう結果を返します」という **決められた窓口**。このアプリのフロントエンドは、バックエンドの `/api/` で始まる窓口に要求を出してデータをやりとりします。**たとえ:** 銀行のATM。「暗証番号を入れて金額を指定すれば、お金が出る」という決まった手順の窓口。中で何が起きているかは知らなくても使える。

エンドポイント（えんどぽいんと：APIの個々の窓口の住所） APIの中の「ここに要求を出すと、これが返ってくる」という個別のURL。たとえば `/api/kosu/` は工数データの窓口、というように。**たとえ:** 役所の「○番窓口」。住民票は3番窓口、印鑑証明は5番窓口、というように用途ごとに窓口が分かれている。

データベース（でーたべーす：データをきちんと整理して保管する倉庫） 大量のデータを、検索・追加・更新しやすい形で保管しておく仕組み。このアプリは **PostgreSQL（ポストグレスキューエル）** というデータベースを使います。略して「DB（ディービー）」。**たとえ:** 巨大な図書館。本（データ）が番号で整理され、すぐ探し出せる。

PostgreSQL（ポストグレスキューエル：このアプリが使うデータベースソフト） 無料で高機能なデータベースソフト。略して「Postgres（ポスグレ）」。従業員データや工数データが、ここに表（テーブル）の形で保管されます。**たとえ:** さっきの図書館の「運営会社」。実際に本を管理しているメーカー名のようなもの。

> **HTTP（エイチティーティーピー：Web上でデータをやりとりする約束ごと）** HyperText Transfer Protocol の略。ブラウザとサーバーが会話するときの「言葉のルール」。リクエストとレスポンスはこのHTTPに従って送受信されます。`https` はそれを暗号化（盗み見されないように）した安全版。**たとえ:** 手紙のルール。「宛名はここ、本文はここ、切手はここ」と決まっているから、郵便屋さんが正しく配れる。
>
> **JSON（ジェイソン：データを文字で表す書き方）** JavaScript Object Notation の略。`{ "name": "山田", "age": 30 }` のように、データを「名前: 値」の形で書き表す方式。フロントとバックは、このJSON形式でデータをやりとりします。**たとえ:** 記入済みの申込用紙。「氏名: 山田」「年齢: 30」と項目ごとに整理されているので、誰が見ても読める。
>
> **URL（ユーアールエル：インターネット上の住所）** `https://example.com/login` のような、ページやデータの場所を示す文字列。「どこに要求を出すか」を表します。**たとえ:** 建物の住所。「○○県○○市○○町1-2-3」と書けば、そこに荷物を届けられる。
>
> **401（よんまるいち：「ログインしていない」を表すエラー番号）** HTTPの応答に付く番号の1つで、「あなたは認証されていません（ログインが必要）」を意味します。このアプリでは、401が返ってきたらフロントが自動で `/login` 画面に飛ばす作りになっています。**たとえ:** 「会員証を見せてください」と入口で止められること。番号で「未認証」と知らせている。

---

3. 開発の道具に関わる言葉

コードを書いたり動かしたりするための「道具」に関わる言葉です。

エディタ（えでぃた：コードを書くためのソフト） プログラムの文章（コード）を書くための専用のソフト。このアプリの開発では **VS Code（ブイエスコード）** がよく使われます。色分け（シンタックスハイライト）や補完機能で、書きやすくしてくれます。**たとえ:** 高機能なワープロ。誤字を指摘してくれたり、言葉を予測変換してくれたりする。

ターミナル（たーみなる：文字でコンピュータに命令する画面） マウスではなく、文字（コマンド）で命令を打ち込む画面。「コマンドプロンプト」「PowerShell」「シェル」とも呼ばれます。`npm start` や `python manage.py runserver` などはここに打ち込みます。**たとえ:** 料理人への口頭指示。「ボタンを押す」のではなく「○○して」と言葉で頼む方式。

コマンド（こまんど：ターミナルに打ち込む1つの命令） ターミナルに打ち込む命令の1行。`npm install` なら「必要な部品を取ってきて」という意味の命令。**たとえ:** 「いらっしゃい」「お会計」のような、店員さんへの決まり文句。

シェル（しぇる：コマンドを受け取って実行する仕組み） ターミナルに打ったコマンドを解釈して実行するプログラム。Windowsの **PowerShell（パワーシェル）** や、Mac/Linuxの **bash（バッシュ）** など。**たとえ:** 注文を聞いて厨房に通す「ホール係」。あなたの言葉を受け取って実行係に渡す。

Node.js（ノードジェイエス：ブラウザの外でJavaScriptを動かす仕組み） 本来ブラウザの中でしか動かないJavaScriptを、パソコン上で直接動かせるようにする土台。フロントエンドの開発やビルドにこれが必要です。このアプリには `node-v22.17.1` というバージョンの記録があります。**たとえ:** 水中でしか泳げない魚（JavaScript）を、陸でも生きられるようにする「水陸両用スーツ」。

npm（エヌピーエム：フロントエンドの部品を管理する道具） Node Package Manager の略。フロントエンド（React側）で使う「部品（パッケージ）」を取ってきたり、開発用の命令を実行したりする道具。`npm install`（部品取得）、`npm start`（開発サーバー起動）、`npm run build`（本番用ビルド）のように使います。**たとえ:** 通販サイト＋家電のリモコン。必要な部品を注文し、アプリの起動ボタンも兼ねている。

pip（ピップ：Pythonの部品を管理する道具） npm のPython版。バックエンド（Django側）で使うPythonの部品を取ってくる道具。`pip install` のように使います。**たとえ:** npm が「フロント用の通販」なら、pip は「バック用の通販」。

パッケージ（ぱっけーじ：再利用できる部品のまとまり） 誰かが作って公開してくれた、便利な機能のまとまり。「ライブラリ」とほぼ同じ意味。自分でゼロから作らず、これを取り込んで使います。React も Django も、巨大なパッケージの一種です。**たとえ:** 料理の「市販のだしパック」。自分でかつお節から取らなくても、これを使えばすぐおいしくなる。

ライブラリ（らいぶらり：便利な機能の部品集） パッケージとほぼ同じ。「自分のコードから呼び出して使う便利機能の集まり」。Chart.js（グラフ描画）、Axios（通信）などがこのアプリのライブラリです。**たとえ:** 道具箱。必要なときに必要な道具（機能）を取り出して使う。

フレームワーク（ふれーむわーく：アプリの骨組みを最初から用意してくれる土台） ライブラリよりさらに大きく、「アプリの作り方の枠組み」ごと提供してくれるもの。React はフロントの、Django はバックのフレームワークです。「こう作りなさい」という流儀があり、それに乗ると速く正確に作れます。**たとえ:** プレハブ住宅のキット。柱・壁・配線の流儀が決まっていて、その通りに組めば家が建つ。ライブラリ（道具）より、もっと大きな「設計の型」。

ビルド（びるど：開発用のコードを、本番で動く形に変換すること） 人間が書いた読みやすいコードを、ブラウザが効率よく実行できる形へまとめ・圧縮する作業。`npm run build` を実行すると `frontend/build/` の中に、本番用の小さく固めたファイルが作られます。**たとえ:** 引っ越しの荷造り。普段使いの形（散らかった部屋）を、運びやすい形（段ボールに圧縮）に変える。

package.json（ぱっけーじどっとジェイソン：フロントの設計図ファイル） フロントエンドが「どの部品を使うか」「どんな命令があるか」を書いた一覧ファイル。`npm start` などの命令の正体もここに書かれています。**たとえ:** レシピの材料表。「この料理にはこの材料がこれだけ要る」と書いてある。

requirements.txt（りくわいやめんつどっとてきすと：バックの部品一覧ファイル） バックエンドが使うPython部品の一覧。`pip install -r requirements.txt` で、ここに書かれた部品を一括で取得できます。**たとえ:** こちらもバックエンド用の「材料表」。

.env（どっとえんぶ：秘密の設定を書くファイル） パスワードや鍵など、コードに直接書きたくない秘密の設定を入れておくファイル。このアプリのバックエンドは `SECRET_KEY` `DB_PASSWORD` などをここに書きます。Gitには載せない（公開しない）のが鉄則。**たとえ:** 金庫。大事な鍵やパスワードを、人目につかない場所にしまっておく。

環境変数（かんきょうへんすう：プログラムの外から与える設定値） プログラムのコードの外側（OSや `.env` ）から渡す設定値。「本番か開発か」「DBのパスワードは何か」などを、コードを書き換えずに切り替えられます。**たとえ:** エアコンの設定温度。本体（コード）はそのままで、外からダイヤル（環境変数）を変えれば動きが変わる。

ローカル（ろーかる：自分のパソコンの中。本番ではない練習場） 「ローカル環境」は、自分のパソコン上で動かす開発・練習用の環境。ここなら壊しても本番に影響しません。`localhost` （ろーかるほすと）は「自分自身」を指す住所で、`http://localhost:3000` は自分のPC内のフロント、`8000` はバックを指します。**たとえ:** 自宅の練習用キッチン。失敗しても店（本番）には出さないので安心。

本番（ほんばん：実際に利用者が使う環境） 「本番環境」は、本物の利用者が実際に使っている環境。このアプリでは Azure 上で動きます（section 11）。ここを壊すと利用者に迷惑がかかるので、慎重に。**たとえ:** 営業中の本物の店舗。お客さんが来ているので、失敗は許されない。

デバッグ（でばっぐ：バグ＝不具合を見つけて直す作業） プログラムの間違い（バグ）を探して直すこと。エラーメッセージを読んだり、途中の値を表示させたりして原因を突き止めます。**たとえ:** 家電が動かないとき、コンセント→ヒューズ→基板と順に調べて故障箇所を見つける作業。

コンソール（こんそーる：プログラムからのメッセージが出る場所） プログラムが「いまこうなってます」「ここでエラーです」と文字で知らせてくる場所。ブラウザではF12の「Console」タブ、バックではターミナルがこれにあたります。**たとえ:** 機械の表示ランプ＆音声案内。「正常です」「エラーです」と教えてくれる。

ログ（ろぐ：出来事の記録） プログラムが動く中で「何が起きたか」を時系列で残した記録。障害が起きたとき、まずこれを見て原因を探します。**たとえ:** 航海日誌（こうかいにっし）。「何時に何が起きた」と書いてあるので、後から追える。

---

4. Git（バージョン管理）の言葉

Git（ギット） は、コードの変更履歴を記録・管理する道具です。「いつ・誰が・どこを・なぜ変えたか」を残し、過去にも戻れます。ここはチーム開発の要なので、用語を厚めに説明します。

バージョン管理（ばーじょんかんり：変更の歴史を記録して、いつでも戻れるようにする仕組み） ファイルの変更を1つずつ記録し、「3日前の状態に戻す」「誰がこの行を変えたか調べる」ができる仕組み。Git がその代表。**たとえ:** ゲームのセーブ機能。要所でセーブしておけば、失敗しても好きなセーブ地点に戻れる。

Git（ギット：いちばん使われているバージョン管理の道具） コードの変更履歴を管理する道具。このアプリも Git で管理されています（リポジトリのルートに `.git` フォルダがある）。**たとえ:** 精密なセーブ＆履歴管理の機械。何を変えたか全部覚えていてくれる。

リポジトリ（りぽじとり：Gitで管理する「プロジェクトの保管庫」） 略して「リポ」。1つのプロジェクトのコードと、その全変更履歴をまとめて入れておく入れ物。このアプリ全体で1つのリポジトリです。**たとえ:** プロジェクト専用の「履歴つき書類棚」。中の全書類の改訂履歴も一緒に保管される。

コミット（こみっと：変更を1つのまとまりとして履歴に刻むこと／その記録1件） 「ここまでの変更を、こういう意味のまとまりとして記録する」操作。1コミットごとに「メッセージ（何をしたか）」を付けます。このアプリの履歴には `休憩変更時の工数消去バグ修正` などのコミットメッセージが並んでいます。**たとえ:** ゲームのセーブ1回。「ボス撃破後」とメモを付けてセーブするイメージ。後で「あのセーブに戻ろう」ができる。

コミットメッセージ（こみっとめっせーじ：そのコミットで何をしたかの説明文） コミットに添える短い説明。「何を・なぜ変えたか」を後で読む人（未来の自分を含む）に伝えます。日本語でわかりやすく書くのが親切。**たとえ:** セーブデータに付けるメモ。「○○を直した」と書いておくと、後で見たとき思い出せる。

ブランチ（ぶらんち：作業を分けるための「枝分かれした作業場」） 本流（みんなが使う安定版）とは別に、自分の作業用に枝分かれさせた作業場。ここで好きに変更し、できたら本流に合流させます。本流は `master`（ますたー）または `main`（めいん）と呼ばれます。**たとえ:** 料理の「別皿で味見」。本鍋（master）に入れる前に、小皿で試作する。失敗しても本鍋は無事。

master / main（ますたー／めいん：中心となる本流のブランチ） プロジェクトの中心・安定版のブランチ。このアプリの本流は `master` です。ここに直接変更を入れず、ブランチで作業してから合流させるのが安全。**たとえ:** 本番に出す「本鍋」。みんなが食べる本物。

チェックアウト／switch（ちぇっくあうと／すいっち：作業するブランチを切り替えること） 「いま作業するブランチを、こっちに変える」操作。作業場を移動するイメージ。**たとえ:** 別の作業台に移動すること。机A（master）から机B（自分のブランチ）へ移る。

ステージング／add（すてーじんぐ／あど：コミットに含める変更を選んでおくこと） たくさん変更したうちの「今回のコミットに入れる分」を選んで準備する操作。`git add` で行います。**たとえ:** 段ボールに荷物を入れる作業。「この箱（コミット）には、この荷物（変更）を入れる」と選り分ける。

プッシュ（ぷっしゅ：手元のコミットを、共有サーバーへ送ること） 自分のパソコンで作ったコミットを、みんなが見られる共有の場所（リモート）へアップロードすること。`git push`。**たとえ:** 撮った写真をクラウドにアップロード。自分のスマホ（ローカル）から共有アルバム（リモート）へ。

プル（ぷる：共有サーバーの最新を、手元へ取り込むこと） 他の人がプッシュした最新の変更を、自分のパソコンへダウンロードして取り込むこと。`git pull`。**たとえ:** 共有アルバムから、他の人がアップした写真を自分のスマホへ取り込む。

フェッチ（ふぇっち：取り込みの前半。最新情報を見るだけ取ってくる） プルの「取ってくるだけ（まだ自分の作業には混ぜない）」部分。`git fetch`。**たとえ:** 共有アルバムの新着を「眺めるだけ」ダウンロード。まだ自分のアルバム

には貼らない。

マージ（まーじ：2つのブランチの変更を合流させること） 枝分かれしていた作業を、本流に合流させること。`git merge` 。**たとえ:** 別皿で作った味付けを、本鍋に入れて混ぜる。

コンフリクト（こんふりくと：合流時にぶつかる「変更の衝突」） 2人が「同じ行」を別々に変えていたとき、Gitが「どっちを採用すればいいか自動で決められない」状態。手で「こっちを残す」と解決します。怖がらず、落ち着いて両方を見比べて選べば大丈夫。**たとえ:** 2人が同じ予定欄に別の予定を書き込んだ手帳。「どっちが正しい？」と人間が判断する必要がある。

GitHub（ギットハブ：Gitのリポジトリを置く共有サービス） Git のリポジトリをインターネット上に置き、チームで共有できるサービス。プッシュ／プルの「共有の場所（リモート）」としてよく使われます。`gh` というコマンドで操作することも。**たとえ:** プロジェクト書類棚を置く「貸し倉庫サービス」。みんながアクセスできる共有の保管場所。

リモート（りもーと：共有サーバー上のリポジトリ） 自分のPC（ローカル）に対して、GitHubなど「向こう側」にある共有のリポジトリ。`origin` （おりじん）という名前が付くのが普通です。**たとえ:** ローカル＝自分の机、リモート＝みんなの共有書庫。

プルリクエスト／PR（ぷるりくえすと：「この変更を本流に入れてください」という提案） 自分のブランチの変更を本流へマージしてもらうための「お願い＆レビュー依頼」。略して「PR」。仲間がコードを確認（レビュー）してから合流させる、安全な合流の手続きです。**たとえ:** 「この企画書、本決定にしてOKですか？」と上司に確認を回す稟議（りんぎ）。承認されたら正式採用。

diff（でぃふ：変更の前後を見比べた差分） 「どの行をどう変えたか」を、追加（緑）・削除（赤）で示したもの。コミット前やレビューで必ず見ます。**たとえ:** 間違い探し。「ここが変わった」と色で教えてくれる。

.gitignore（どっとぎっといぐのあ：Gitで管理しないものを指定するファイル） 「これは履歴に入れないで」というファイルやフォルダを並べたリスト。`.env` （秘密情報）や巨大な生成物などを除外します。**たとえ:** 「これは倉庫に入れないでね」という持ち込み禁止リスト。

---

5. HTML・CSS の言葉

画面の「骨組み（HTML）」と「見た目（CSS）」に関わる言葉です。詳しくは第3章で扱っています。ここは引きやすいように要点だけまとめます。

HTML（エイチティーエムエル：画面の骨組みを作る言語） HyperText Markup Language の略。`<p>` `<div>` などの **タグ** で、文章の構造（見出し・段落・表・ボタン）を表す言語。**たとえ:** 家の「骨組み（柱・壁）」。形を決める。

CSS（シーエスエス：画面の見た目を整える言語） Cascading Style Sheets の略。色・大きさ・余白・配置など「見た目」を指定する言語。**たとえ:** 家の「内装・塗装」。骨組み（HTML）に化粧をする。

タグ（たぐ：HTMLの「ここからここまでは○○」という印） `<p>...</p>` のように、文章に「これは段落」「これは見出し」と意味のラベルを貼る印。**たとえ:** 荷物に貼る「割れ物注意」「天地無用」のシール。中身の扱い方を示す。

属性（ぞくせい：タグに足す追加情報） `<input type="number">` の `type="number"` のように、タグに「どんな種類か」「リンク先はどこか」を足す情報。**たとえ:** シールに書く補足メモ。「割れ物注意（特にガラス）」の「特にガラス」の部分。

要素（ようそ：開始タグ＋中身＋終了タグのひとまとまり） `<p>本文</p>` 全体で1つの「要素」。HTMLは要素の入れ子で組み立てます。**たとえ:** 1つの「部品」。柱1本、壁1枚のような単位。

DOM（ドム：HTMLをプログラムから操作できる木構造にしたもの） Document Object Model の略。ブラウザがHTMLを読み込んで作る、操作可能な「木構造（ツリー）」。Reactはこれを効率よく書き換えて画面を更新します。**たとえ:** 家の「立体模型」。紙の設計図（HTML文字列）を、手で動かせる模型（DOM）にしたもの。模型の一部を動かせば、実物（画面）も変わる。

レンダリング（れんだりんぐ：指示を実際の見た目として画面に描くこと） 「色は赤、ここに配置」という指示を、本物のピクセル（点の集まり）として画面に描き出すこと。「描画」とも。**たとえ:** 設計図をもとに、実際にペンキを塗り家具を置いて部屋を仕上げる作業。

セレクタ（せれくた：CSSで「どの要素を飾るか」を指定する部分） `.blue_button { ... }` の `.blue_button` のように、「この見た目を、どの要素に当てるか」を選ぶ書き方。**たとえ:** 「赤い帽子の人」と人を指定する言い方。条件に合う相手を選び出す。

クラス（くらす：複数の要素で見た目を共有するための名札） `class="blue_button"` の名札。同じ名札を付けた要素は、同じ見た目になります。CSSでは `.名前` で狙います。**たとえ:** 「制服」。同じ制服（クラス）を着た人は、同じ見た目になる。

ID（アイディー：1ページに1つだけの固有の名札） `id="numberInput"` のように、ページ内で1つだけ付ける固有名。CSSでは `#名前` で狙います。**たとえ:** 「学籍番号」。1人に1つの固有の番号。

ボックスモデル（ぼっくすもでる：すべての要素は四角い箱だという考え方） HTMLの全要素を「中身（content）＋内側余白（padding）＋枠線（border）＋外側余白（margin）」の四重の箱として扱う考え方。CSSで最重要。**たとえ:** プレゼント。中身（content）→緩衝材（padding）→箱（border）→箱と箱の間の隙間（margin）。

margin（まーじん：箱の「外側」の余白。他の箱との距離） 要素と要素の「間」のすき間。`margin: 0 auto` は「左右の余白を自動均等＝横中央寄せ」の定番。**たとえ:** 家と家の間の「敷地の境界の空き地」。

padding（ぱでぃんぐ：箱の「内側」の余白。枠と中身の間） 枠線と中身の「内側」のすき間。ボタンの厚み感はこれで作ります。**たとえ:** 箱の内側に詰める緩衝材。中身（文字）と箱（枠）の間のゆとり。

border（ぼーだー：枠線） 要素のまわりの線。`border: 1px solid #ddd` は「1pxの実線、薄い灰色」。**たとえ:** 箱そのものの「壁」。

box-sizing（ぼっくすさいじんぐ：幅の計算方法の指定） `box-sizing: border-box` にすると、`width` が「padding と border を含めた幅」になり、計算が直感どおりになります。このアプリは全要素にこれを設定済み。**たとえ:**「服のサイズ表記」。中身だけ測る方式か、肩パッド込みで測る方式かの違い。border-box は「全部込みのサイズ」。

Flexbox（ふれっくすぼっくす：要素を横並び・縦並び・中央寄せする仕組み） 親に `display: flex` を付けると、子要素を柔軟に並べられる現代CSSの主役。`justify-content`（主軸の揃え）、`align-items`（交差軸の揃え）、`gap`（すき間）で配置を決めます。**たとえ:** 棚の整理。「左寄せ／中央／等間隔に並べる」を指示できる、賢い陳列棚。

flex-direction（ふれっくすだいれくしょん：並ぶ向き） `row`（横並び・既定）か `column`（縦並び）。このアプリのログイン入力欄は `column` で縦に並びます。**たとえ:** 商品を「横一列」に並べるか「縦一列」に並べるかの指定。

justify-content / align-items（じゃすてぃふぁいこんてんつ／あらいんあいてむず：Flexの揃え方） `justify-content` は並ぶ向き（主軸）の揃え、`align-items` は直角方向（交差軸）の揃え。`center` で中央に集められます。**たとえ:** 集合写真。横の位置（前者）と縦の位置（後者）をそれぞれ指示する。

擬似クラス（ぎじくらす：要素の「状態」に応じた見た目。`:` が1つ） `:hover`（マウスを乗せた時）、`:focus`（入力欄を選択中）など。状態でスタイルを変えます。**たとえ:**「触れたら光るボタン」。普段と、触れた時で見た目が変わる。

擬似要素（ぎじようそ：要素の前後に飾りの中身を作る。`::` が2つ） `::before` `::after` で、要素の中身の前後に文字や飾りを足します。`content` プロパティで中身を指定。このアプリのアラートの「⚠️」やヘルプボタンの「?」はこれで作っています。**たとえ:** 名札に自動で付く「★」マーク。本体（要素）とは別に、飾りを足す。

CSS変数（しーえすえすへんすう：よく使う値に名前を付けて使い回す仕組み）「カスタムプロパティ」とも。`:root { --heading1: ...; }` で定義し、`var(--heading1)` で呼び出します。1か所直せば全体に反映。**たとえ:**「テーマカラー」の登録。色を1か所で決めておけば、全ページに一括反映できる。

clamp（くらんぷ：値を最小〜最大の範囲に収める関数） `clamp(最小, 推奨, 最大)`。画面サイズに応じて文字を伸縮させつつ、小さすぎ・大きすぎを防ぎます。**たとえ:**「身長に合わせて伸びるが、伸びすぎない突っ張り棒」。範囲内で自動調整。

calc（かるく：CSSの中で計算する関数） `calc(100% - 60px)` のように、CSSの値を計算式で書ける関数。「親の幅いっぱいから60px引いた幅」など。**たとえ:** その場で電卓をたたく。「全体から少し引いた長さ」を計算してくれる。

rem / px / vw / %（れむ／ぴくせる／ぶいだぶりゅー／ぱーせんと：大きさの単位） `px` は絶対的な点、`%` は親に対する割合、`rem` は基準文字サイズ基準、`vw` は画面幅の1%。用途で使い分けます。**たとえ:** 長さの単位（cm/インチ/歩幅）。測り方が違うだけで、どれも「大きさ」を表す。

メディアクエリ（めでぃあくえり：画面幅に応じてスタイルを切り替える仕組み） `@media (max-width: 768px) { ... }` で「画面幅768px以下のときだけ」効くスタイルを書けます。PC・タブレット両対応に使います。**たとえ:**「晴れの日は半袖、雨の日は長袖」と服を切り替える条件分け。

レスポンシブ（れすぽんしぶ：画面サイズに応じて見やすく変わる設計） 1つのコードで、大きい画面でも小さい画面でも見やすく表示を変える設計。メディアクエリや `clamp` で実現。**たとえ:**「伸び縮みする服」。体型（画面）が変わってもフィットする。

CSS Modules（しーえすえすもじゅーるず：CSSの効く範囲を1画面に限定する仕組み） `○○.module.css` という名前にすると、そのCSSがその画面だけに効くようになり、別画面との名前衝突を防げます。`className={styles["名前"]}` の形で使います。**たとえ:**「学校ごとの校則」。同じ「制服」という言葉でも、A校とB校で別物として扱われ、混ざらない。

JSX（ジェイエスエックス：JavaScriptの中にHTMLのような記述を混ぜる書き方） Reactで画面を書く記法。HTMLとほぼ同じだが、`class` → `className` 、`for` → `htmlFor` 、`{ }` で値を埋め込む、の3点が違います。詳しくはsection 7。**たとえ:**「日本語に英単語を混ぜた文章」。基本は片方の文法だが、もう片方を埋め込める。

---

6. JavaScript・TypeScript の言葉

画面に「動き」を与える言語です。詳しくは第4章で扱います。ここは引きやすく要点を。

JavaScript（じゃばすくりぷと：ブラウザを動かす定番の言語） 略して「JS」。ボタンを押したら反応する、入力を検証する、サーバーと通信する、といった「動き」を担う言語。**たとえ:** 家の「電気・設備」。スイッチを押すと電気がつく、その配線部分。

TypeScript（たいぷすくりぷと：JavaScriptに「型」を足した言語） 略して「TS」。JavaScriptに「この変数は数値」「これは文字列」という **型（かた）** の情報を足し、間違いを書く前に気づけるようにした言語。このアプリのフロントはTSで書かれています（ `.tsx` ファイル）。**たとえ:** JavaScriptに「指差し確認」を足したもの。「これは数値ですね？」と事前に念押ししてくれる。

型（かた：そのデータが「何の種類か」の情報） 「数値」「文字列」「真偽（true/false）」など、データの種類。TypeScriptは型を見て、間違った使い方（数値に文字列を足すなど）を警告します。**たとえ:** コンセントの形。丸い穴には丸いプラグしか挿せない。形（型）が合わないと使えない、と事前に教えてくれる。

変数（へんすう：値に付けた名前の入れ物） `const name = "山田"` の `name` のように、値を入れて名前を付けた箱。後でその名前で値を取り出せます。`const`（変更しない）と `let`（変更する）があります。**たとえ:** ラベルを貼った保存容器。「砂糖」と書いた容器に砂糖を入れておけば、後で「砂糖」と言えば取り出せる。

関数（かんすう：処理に名前を付けてまとめ、何度でも呼べるようにしたもの） 「入力を受け取り、何か処理して、結果を返す」ひとまとまりに名前を付けたもの。`handleSubmit` のように。呼べば中の処理が動きます。**たとえ:** 自動販売機。お金（入力）を入れてボタンを押すと、飲み物（結果）が出る。中の仕組みは知らなくても、入れて押せば使える。

引数（ひきすう：関数に渡す入力の値） 関数を呼ぶときに渡す材料。`Number(employee_no)` の `employee_no` が引数。**たとえ:** 自動販売機に入れる「お金」と「押すボタン」。

戻り値／返り値（もどりち／かえりち：関数が返す結果） 関数が処理の結果として返す値。`Number("5")` は `5`（数値）を返します。**たとえ:** 自動販売機から出てくる「飲み物」。

オブジェクト（おぶじぇくと：複数の値を「名前: 値」でまとめたもの） `{ name: "山田", age: 30 }` のように、関連する値をひとまとめにしたデータ。**たとえ:** 名刺。「氏名・会社・電話番号」を1枚にまとめてある。

配列（はいれつ：値を順番に並べたリスト） `[1, 2, 3]` や `["P", "R", "W1"]` のように、複数の値を順番に並べたもの。**たとえ:** 番号付きの下駄箱。0番、1番、2番…と順に値が入る。

条件分岐（じょうけんぶんき：「もし○○なら」で処理を分けること） `if (条件) { ... }` で「条件が成り立つときだけ」処理する仕組み。`07` 章の障害対応でもよく出ます。**たとえ:** 分かれ道。「晴れなら公園へ、雨なら室内へ」と進路を分ける。

ループ（るーぷ：同じ処理を繰り返すこと） `for` などで、同じ処理を何回も繰り返す仕組み。配列の全要素を順に処理するときなどに使います。**たとえ:** ベルトコンベア。同じ作業を流れてくる品物すべてに繰り返す。

非同期（ひどうき：結果を待たずに次へ進み、できたら受け取る方式） 通信のように「結果が返るまで時間がかかる処理」を、待ちっぱなしにせず、他の処理を進めつつ、できたら受け取る方式。**たとえ:** 食堂で番号札をもらって席で待つ方式。料理ができるまで突っ立って待たず、座って他のことをし、呼ばれたら取りに行く。

Promise（ぷろみす：「あとで結果を渡すね」という約束を表すもの） 非同期処理の結果を表すオブジェクト。「いま結果はないけど、終わったら渡す（成功か失敗か）」という **約束** を持っています。**たとえ:** 食堂の番号札。「この番号で、できたら料理（結果）をお渡しします」という引換券そのもの。

async / await（えいしんく／あうぇいと：非同期を読みやすく書く書き方） `async` を付けた関数の中で `await` を使うと、「ここで結果が出るまで待ってから次へ」と、まるで順番どおりのように書けます。Promiseを読みやすくする道具。**たとえ:** 番号札（Promise）を「呼ばれるまで待つ（await）」と素直に書ける書き方。複雑な約束を、順番の物語として読める。

コールバック（こーるばっく：「終わったら呼んでね」と渡しておく関数） 「この処理が終わったら、この関数を呼んで」と、あらかじめ渡しておく関数。ボタンの `onClick` に渡す処理などがこれ。**たとえ:** 「料理ができたら、この番号に電話して」と渡しておく連絡先。

イベント（いべんと：ユーザー操作などの「出来事」） クリック、入力、送信などの「出来事」。`onClick`（クリック時）、`onChange`（入力変化時）、`onSubmit`（送信時）で、その出来事に反応する処理を結びつけます。**たとえ:** 「ドアが開いたら鳴るチャイム」。出来事（ドアが開く）に反応（チャイム）を結びつける。

null / undefined（ぬる／あんでぃふぁいんど：「値がない」状態） `null` は「意図的に空」、`undefined` は「まだ値が入っていない」状態。両方とも「中身がない」を表し、これらを操作しようとするとエラーの原因になりがち。**たとえ:** 空っぽの箱。`null` ＝わざと空にした箱、`undefined` ＝まだ何も入れていない箱。

真偽値／boolean（しんぎち／ぶーりあん：trueかfalseの2択の値） `true`（はい・真）か `false`（いいえ・偽）のどちらか。「チェックが入っているか」などを表します。**たとえ:** スイッチのオン／オフ。2つの状態しかない。

文字列（もじれつ：文字を並べたデータ） `"山田太郎"` のような文字の並び。引用符（`"` や `'`）で囲みます。**たとえ:** 文字を書いた紙テープ。

数値（すうち：数のデータ） `30` や `3.14` のような数。計算ができます。`<input type="number">` から来る値は実は文字列なので、`Number()` で数値に変換することがあります（このアプリのログイン処理がその例）。**たとえ:** 電卓に入れられる数。文字列の `"30"` は計算できないが、数値の `30` は計算できる。

---

7. React の言葉

フロントエンドのフレームワーク **React（リアクト）** に関わる言葉です。詳しくは第5章。ここは引きやすく厚めに。

React（りあくと：画面を「部品」の組み合わせで作るフレームワーク） 画面を **コンポーネント（部品）** に分け、それらを組み合わせて作るフロントエンドのフレームワーク。データが変わると、関係する部分だけ自動で画面を更新してくれます。このアプリのフロントの本体。**たとえ:** レゴブロック。小さな部品（コンポーネント）を組み合わせて、大きな作品（画面）を作る。

コンポーネント（こんぽーねんと：画面を構成する再利用できる部品） ボタン、表、入力欄、ページ全体など、「画面の部品」。1つの関数として書かれ、`<Login />` のようにタグの形で使えます。このアプリの `KosuList`、`MemberList` などがコンポーネント。**たとえ:** レゴの1ピース。組み合わせて使えるし、同じピースを何度でも使える。

props（ぷろっぷす：親コンポーネントから子へ渡す値） 「プロパティ（properties）」の略。親の部品から子の部品へ渡す **設定値・データ**。子は受け取ったpropsを表示や処理に使います。一方通行（親→子）で、子からは変更しません。**たとえ:** 親から子へのお使いメモ。「牛乳を買ってきて」と渡す。子はメモ通りに動くが、メモ自体は書き換えない。

state（すてーと：コンポーネントが内部で持つ「移ろう状態」） 「状態」。入力欄の今の値、開いている/閉じている、エラーメッセージの有無など、時間とともに変わるデータ。stateが変わると、その部品の画面が自動で更新されます。**たとえ:** 電卓の「いま表示中の数字」。ボタンを押すたびに変わり、その変化が画面に出る。

useState（ゆーずすてーと：stateを作るための仕組み） Reactでstateを用意する関数。`const [値, 値を変える関数] = useState(初期値)` の形。`setNumber` のような「set〜」がstateを更新する関数です。**たとえ:** 表示窓と、その窓を書き換えるダイヤルのセット。「窓（値）」と「ダイヤル（set関数）」が一緒に手に入る。

useEffect（ゆーずえふぇくと：画面の表示後などに「副作用」を実行する仕組み） 「副作用（さよう）」＝画面表示以外の処理（サーバーからデータ取得、タイマー設定など）を、適切なタイミングで実行する仕組み。「画面が表示されたらデータを取りに行く」がこのアプリでの典型用途。**たとえ:** 「部屋に入ったら自動で電気をつける」センサー。表示（入室）をきっかけに、別の処理（点灯）を起こす。

フック（ふっく：Reactの機能を使うための「use〜」の関数たち） `useState`、`useEffect` などの「use」で始まる関数の総称。Reactの便利機能に引っかけて使うので「フック（引っかけ）」と呼ばれます。**たとえ:** 道具を引っかける「フック（壁掛けフック）」。Reactの機能を引っかけて取り出す口。

React（再）レンダリング（れんだりんぐ：Reactが画面を作り直すこと） CSSの「描画」と少し意味が違い、Reactでは「stateやpropsをもとに、表示すべき内容を計算し直すこと」を指します。stateが変わると再レンダリング（作り直し）が起きます。**たとえ:** 数字が変わるたびに表示を計算し直す電光掲示板。

仮想DOM（かそうどむ：Reactが裏で持つDOMの下書き） Reactが内部に持つ「DOMの下書き（コピー）」。変更前後の下書きを比べ、本当に変わった部分だけを実際のDOMに反映するので、速い。**たとえ:** 模様替えの「下書き図面」。実際に家具を全部動かす前に、図面上で差分を確認し、動かす必要がある家具だけ動かす。

SPA（エスピーエー：ページを丸ごと読み込み直さず画面を切り替えるWebアプリ） Single Page Application（シングル・ページ・アプリケーション）の略。最初に1回だけページを読み込み、以降は **必要な部分だけJavaScriptで書き換える** ことで、ページ遷移をなめらかにする作り。このアプリもSPAです。**たとえ:** 1枚の大きなホワイトボード。毎回新しいボードに貼り替えるのではなく、同じボードの一部だけ消して書き直す。だから速くてちらつかない。

ルーティング／React Router（るーてぃんぐ／りあくとるーたー：URLに応じて表示する画面を切り替える仕組み） 「`/login` ならログイン画面、`/kosu` なら工数画面」のように、URLに応じて表示するコンポーネントを切り替える

仕組み。このアプリは React Router DOM を使い、`index.tsx` で設定しています。**たとえ:** 受付の案内係。「○○課はこちら」とURLごとに行き先（画面）を案内する。

Link（りんく：SPA内でページを切り替えるためのリンク部品） React Routerの `<Link to="/help">` のような部品。普通の `<a>` と違い、ページ全体を読み込み直さずに画面だけ切り替えます。**たとえ:** 同じホワイトボード内での「ページめくり」。新しい紙を持ってこず、ボード上で表示を切り替える。

Axios（あくしおす：サーバーと通信するためのライブラリ） フロントエンドからバックエンドのAPIへ、リクエストを送りレスポンスを受け取るためのライブラリ。このアプリの通信はAxiosで行われます。**たとえ:** 厨房と客席を行き来する「配膳ロボット」。注文を厨房（API）に運び、料理（データ）を席（画面）に運ぶ。

Material-UI / MUI（まてりある・ゆーあい：見た目の整った部品集） ボタン・表・ダイアログなど、デザインの整った部品をまとめて提供するライブラリ。このアプリのフロントで使われています。**たとえ:** 既製品の高級家具セット。自分で削り出さなくても、整ったパーツが手に入る。

Chart.js（ちゃーとじぇいえす：グラフを描くライブラリ） 棒グラフ・円グラフなどを描くためのライブラリ。工数の集計をグラフで見せる画面で使われます。**たとえ:** グラフ専門の作画担当。数字を渡すと、見やすいグラフに描いてくれる。

FullCalendar（ふるかれんだー：カレンダー表示のライブラリ） 月間カレンダーや予定表を表示するライブラリ。日付ごとの工数を見せる画面などで使われます。**たとえ:** 壁掛けカレンダー部品。日付のマス目を自動で組んでくれる。

ページネーション（ぺーじねーしょん：たくさんの行を「○件ずつのページ」に分けて表示する仕組み） 一覧画面で、全件を一度に出さず「1ページ分ずつ」表示し、ページをめくって続きを見られるようにする仕組み。このアプリには共通の `Pagination` 部品があります（バックエンド側の説明はsection 8にも）。**たとえ:** 分厚い名簿の「ページ送り」。1ページ見終わったら次のページへ。

---

8. Python・Django の言葉

バックエンドの言語 **Python（パイソン）** と、そのフレームワーク **Django（ジャンゴ）** に関わる言葉です。詳しくは第6章。ここは特に厚く説明します。

Python（ぱいそん：読みやすさを重視した定番のプログラミング言語） バックエンドを書いている言語。英語に近い読みやすい文法が特徴。**たとえ:** きれいに整理された日本語マニュアル。読みやすさを優先して書かれている。

Django（じゃんご：Pythonでバックエンド（Webアプリの裏側）を作るフレームワーク） データベース操作・URLの割り振り・認証など、Webアプリの裏側に必要なものが一式そろったフレームワーク。このアプリのバックエンドの本体。`manage.py` というファイルから各種操作をします。**たとえ:**「役所の業務一式キット」。受付・台帳管理・本人確認の仕組みが最初からそろっている。

Django REST Framework / DRF（じゃんご・れすと・ふれーむわーく：DjangoでAPIを作る拡張） Djangoで、JSONをやりとりするAPIを作りやすくする追加部品。このアプリの `/api/` の窓口はこれで作られています。**たとえ:** 役所キットに足す「外部問い合わせ窓口セット」。外部（フロント）とのデータ受け渡しを楽にする。

モデル（もでる：データベースの表を、Pythonのクラスで表したもの）「このデータはこういう項目を持つ」という設計図を、Pythonのクラスとして書いたもの。`models.py` に並びます。このアプリの `member`（従業員）、`Business_Time_graph`（工数記録）などがモデルです。1つのモデルが、データベースの1つの **テーブル（表）** に対応します。**たとえ:** 台帳の「様式（フォーマット）」。「氏名・従業員番号・ショップ…」と、どんな項目を記録するかを定めた用紙のひな形。

テーブル（てーぶる：データベースの中の1つの表） データを行と列で並べた表。列が項目（氏名・従業員番号…）、行が1件1件のデータ。1つのモデルが1つのテーブルに対応します。**たとえ:** Excelの1シート。1行＝1人分、列＝項目。

フィールド（ふぃーるど：モデル（テーブル）の1つの項目（列）） `employee_no = models.IntegerField('従業員番号')` のように、モデルの中の1項目。`IntegerField`（整数）、`CharField`（文字列）、`DateField`（日付）、`BooleanField`（はい/いいえ）など、項目の種類を指定します。**たとえ:** 台帳の1つの「欄」。「氏名欄」「従業員番号欄」のそれぞれ。

ORM（オーアールエム：SQLを書かずにPythonでデータベースを操作する仕組み） Object-Relational Mapping の略。本来データベースは **SQL** という専用言語で操作しますが、ORMを使うとPythonのコードで操作でき、SQLを直接書かなくて済みます。Djangoの大きな魅力。**たとえ:** 自動翻訳機。あなたが日本語（Python）で頼むと、機械が外国語（SQL）に訳してデータベースに伝えてくれる。

SQL（エスキューエル：データベースを操作する専用言語） `SELECT * FROM member` のように、データベースに「取り出して」「保存して」と指示する専用言語。ORMがあるので普段は直接書きませんが、裏ではこれが動いています。**たとえ:** 図書館司書への専門用語での指示。ORM（翻訳機）があれば、普段の言葉で頼める。

QuerySet（くえりせっと：データベースから取り出した「行の集まり」） モデルを使って「条件に合うデータを取り出した結果」。複数件の行のまとまりです。`member.objects.filter(shop='P')` は「ショップがPの従業員たち」というQuerySetを返します。実際にデータベースへ問い合わせるのは「結果を使う瞬間まで遅らせる（遅延評価）」のが特徴。**たとえ:** 図書館で「ミステリー小説を全部出して」と頼んだときに出てくる「本の山」。条件に合う本のまとまり。

objects（おぶじぇくつ：モデルからデータを取り出すための入口） `member.objects.all()` の `objects`。モデルに付いている「データを取りに行く窓口」。`.all()`（全部）、`.filter()`（絞り込み）、`.get()`（1件取得）などを続けます。**たとえ:** 台帳係さん。「全部見せて（all）」「Pショップだけ（filter）」と頼む相手。

filter / get / all（ふぃるたー／げっと／おーる：データの取り出し方） `.all()` は全件、`.filter(条件)` は条件に合う複数件、`.get(条件)` は条件に合う1件。よく使う取り出し方です。**たとえ:**「全部出して」「赤い本だけ出して」「あの1冊を出して」の頼み方の違い。

マイグレーション（まいぐれーしょん：モデルの変更を実際のデータベースに反映する仕組み） モデル（設計図）を変えたら、その変更を本物のデータベースの表に反映する必要があります。その手順がマイグレーション。`python manage.py makemigrations`（変更内容を記録）→ `python manage.py migrate`（実際に反映）の2段階。**たとえ:** 設計図を直したら、実際の建物も工事して合わせること。makemigrations＝工事計画書づくり、migrate＝実際の工事。

シリアライザ（しりあらいざ：モデルのデータとJSONを相互に変換する係） Pythonのモデルデータ（オブジェクト）を、フロントに送れるJSON（文字）に変換したり、その逆をしたりする係。DRFの中心部品。**たとえ:** 翻訳・梱包係。社内の台帳データ（モデル）を、外部に送れる荷物（JSON）に包み直す。逆に、届いた荷物を台帳の形に戻すこともする。

ビュー（びゅー：リクエストを受けて処理し、レスポンスを返す関数／クラス）「この窓口に要求が来たら、こう処理してこう返す」を書く部分。このアプリでは `kosu/views/` の中に、機能ごとに分けて書かれています（`kosu_views`、`member_views` など）。**たとえ:** 窓口の担当者。要求（リクエスト）を受け、台帳を調べ、回答（レスポンス）を返す。

URLルーティング／urls.py（ユーアールエルるーてぃんぐ：どのURLをどのビューにつなぐか決める設定）「`/api/kosu/` に来たら、この工数ビューへ渡す」という対応表。`kosu/urls.py` に書かれています。**たとえ:** 電話交換台。「営業部につないで」と言われたら、その内線（ビュー）につなぐ。

認証（にんしょう：「あなたは誰か」を確かめること） ログインで「この人は本物の従業員か」を確かめる仕組み。このアプリは **セッション認証** を使います。**たとえ:** 受付での本人確認。社員証を見せて「この人は社員だ」と確かめる。

セッション（せっしょん：ログイン状態を覚えておく仕組み） 一度ログインすると、サーバーが「この人はログイン済み」と覚えておき、その後の要求では毎回パスワードを聞かない仕組み。ブラウザには「合言葉（セッションID）」がクッキーとして保存されます。**たとえ:** 遊園地の再入場スタンプ。最初に入場確認したら、手にスタンプ（セッション）を押し、以降はスタンプを見せれば再入場できる。

クッキー（くっきー：ブラウザがサーバーから預かって保存する小さなデータ） サーバーがブラウザに「これを持っておいて」と渡す小さなメモ。セッションIDの保存などに使われます。次の要求のとき自動でサーバーに送り返されます。**たとえ:** クロークの「引換札」。預けた荷物（ログイン状態）と引き換えるための小さな札。

CSRF（シーエスアールエフ：なりすまし送信を防ぐ仕組み） Cross-Site Request Forgery（クロスサイト・リクエスト・フォージェリ）の略。「別の悪意あるサイトが、ログイン中のあなたになりすまして勝手にデータを送信する」攻撃を防ぐ仕組み。送信時に専用の「合言葉（CSRFトークン）」を一緒に送らせて、本物かを確かめます。**たとえ:** 銀行振込の「ワンタイムパスワード」。本人しか知らない合言葉を一緒に送らせ、なりすましを防ぐ。

CORS（コルス：別ドメインからのアクセスを許可・制限する仕組み） Cross-Origin Resource Sharing（クロスオリジン・リソース・シェアリング）の略。「フロント（`localhost:3000`）から、別の住所のバック（`localhost:8000`）へアクセスしていいか」を制御する仕組み。許可していない相手はブラウザがブロックします。**たとえ:** マンションの入館許可リスト。「この人（このサイト）は入っていい」とあらかじめ登録された相手だけ通す。

デコレータ（でこれーた：関数の上に付けて機能を足す `@〇〇` の印） `@login_required` のように、関数の真上に `@` で付けて、その関数に機能（ログイン必須など）を追加する仕組み。中身を書き換えずに、機能をまとう。

たとえ: 服の上に羽織るコート。本体（関数）はそのままで、上から機能（防寒＝認証チェック）を着せる。

シグナル（しぐなる：「○○が起きたら△△する」を自動で結びつける仕組み） 「データが保存されたら、自動でこの処理を呼ぶ」のように、出来事に処理を結びつけるDjangoの仕組み。このアプリは `signals.py` でモデルの変更を追跡（履歴記録）しています。**たとえ:** 玄関の人感センサー。「人が来た（出来事）ら、自動でライトをつける（処理）」と結びつけておく。

ページネーション（ぺーじねーしょん：大量のデータを「○件ずつのページ」に分けて返す仕組み） 1万件あるデータを一度に全部返すと重いので、「1ページ20件ずつ」に分けて返す仕組み。フロントはページをめくって続きを取得します。**たとえ:** 分厚い本のページ分け。一度に全ページ渡さず、見ているページだけ手渡す。

WSGI（ダブリュエスジーアイ：PythonのWebアプリとサーバーをつなぐ約束ごと） Web Server Gateway Interface の略。Djangoのアプリと、それを動かす本番用サーバー（Gunicornなど）の「つなぎ目の規格」。`hozen_another/wsgi.py` がその入口です。**たとえ:** 電源プラグの規格。アプリ（家電）とサーバー（コンセント）が、決まった形でつながるための差し込み口の標準。

Gunicorn（ジーユニコーン／ガニコーン：本番でDjangoを動かすサーバー） 開発中は `manage.py runserver` で動かしますが、本番ではより頑丈な **Gunicorn** がDjangoを動かします。たくさんの利用者からの要求を、複数の作業員（ワーカー）でさばきます。**たとえ:** 練習用の手回し発電機（runserver）に対し、本番用の業務用発電機（Gunicorn）。たくさんの電力（アクセス）に耐えられる。

WhiteNoise（ほわいとのいず：静的ファイルを配信する部品） CSSや画像などの「変化しないファイル（静的ファイル）」を、Django自身が効率よく配る仕組み。このアプリはこれを使っています。**たとえ:** 受付脇のパンフレット置き場。毎回作らず、用意済みのもの（CSS・画像）をサッと渡す。

静的ファイル／staticfiles（せいてきふぁいる：変化しないファイル＝CSS・JS・画像など） 利用者ごとに変わらない、決まったファイル（CSS、JavaScript、画像）。`python manage.py collectstatic` で `staticfiles/` フォルダに集められ、配信されます。**たとえ:** 店の固定メニュー表。お客さんごとに変わらない、用意済みの配り物。

Django-Q（じゃんごきゅー：時間のかかる処理を裏でこなす仕組み） バックアップ・復元など「時間がかかる処理」を、利用者を待たせず裏で実行する仕組み。`python manage.py qcluster` で起動する作業員が、順番にこなします。このアプリの `tasks.py` にその処理が書かれています。**たとえ:** クリーニングの預かり受付。時間がかかる作業（洗濯）はその場でやらず、預かって裏で進め、できたら返す。

manage.py（まねーじどっとぱい：Djangoのいろいろな操作の入口） `python manage.py runserver`（起動）、`migrate`（DB反映）、`makemigrations`（変更記録）など、Djangoの各種操作を実行するファイル。**たとえ:** リモコン。1つのリモコンから「起動」「設定変更」など各操作のボタンを押せる。

settings.py（せってぃんぐすどっとぱい：Djangoの設定をまとめたファイル） データベース接続先、許可するアクセス元、秘密鍵の読み込みなど、アプリ全体の設定を書くファイル。`hozen_another/settings.py` にあります。**たとえ:** アプリの「設定画面」。Wi-Fi・音量・言語などの設定を1か所にまとめた場所。

9. このアプリ特有の言葉

このアプリ「業務工数システム」だけで使う、業務独自の言葉です。**ここが一番大事**かもしれません。コードを読むとき、この意味を知らないと何のことかわからないからです。実際のモデル（`kosu/models.py`）に基づいて説明します。

工数（こうすう：作業にかかった時間・人手の量） このアプリの主役。「ある作業に、どれだけの時間・人手をかけたか」を表す量。従業員が「いつ・どの作業を・どれだけ」やったかを記録するのが、このアプリの目的です。**たとえ:** 作業の「燃費メーター」。何にどれだけのエネルギー（時間）を使ったかを測る。

member（めんばー：従業員1人を表すモデル） 従業員（社員）1人分のデータを表すモデル。従業員番号・氏名・所属ショップ・権限・各種休憩時間などを持ちます（ `models.py` で定義）。**たとえ:** 社員名簿の1行。1人分の基本情報がまとまっている。

employee_no（じゅうぎょういんばんごう：従業員を識別する番号） 従業員1人ひとりに割り振られた番号。ログイン時にもこの番号を使います。`<input type="number">` で入力するため、届く値は文字列で、`Number()` で数値に直すことがあります。**たとえ:** 社員番号。名前が同じ人がいても、番号で必ず区別できる。

ショップ（しょっぷ：従業員の所属区分） 従業員がどの区分に属するかを表す分類。`models.py` の `shop_list` に `P`、`R`、`W1`、`W2`、`T1`、`T2`、`A1`、`A2`、`J`、`その他`、`異動・退社` などが定義されています。一覧画面では並び順が工夫されています（最近のコミットで「異動・退社を後ろに」変更）。**たとえ:** 部署・班の区分。「営業1課」「製造2班」のような所属の仕分け。

直（ちょく：勤務の時間帯の区分。シフトの種類） モデルの `tyoku2`（直）。1直（昼の勤務）、2直、3直、常昼（じょうひる）、連1直・連2直など、**どの時間帯の勤務か** を表す区分。休憩時間も直ごとに違うため、従業員モデルは直ごとの休憩時間欄をたくさん持っています（ `break_time1` 〜 `break_time6` と、それぞれの残業休憩）。**たとえ:** 工場の交替勤務の「シフト名」。「日勤」「夜勤」「準夜勤」のような、時間帯の呼び名。

288枠（にひゃくはちじゅうはちわく：1日を5分刻みで区切った作業内容の枠） モデルの `time_work`（作業内容）は `max_length=288` の文字列です。これは **1日（24時間）を5分ごとに区切ると 24×60÷5＝288 個の枠** になることに対応します。つまり「1日の各5分が、どの作業だったか」を288個の枠で表す設計です。工数を細かく記録するための、このアプリの核心的な仕組み。**たとえ:** 1日を288マスに区切った塗り絵。各5分のマスを「この作業の色」で塗っていくと、1日の作業の内訳が一目でわかる。

> **なぜ288なの？** 24時間 = 1440分。これを5分単位で割ると 1440 ÷ 5 = 288。だから「5分刻みで1日を表すと288枠」になります。1枠＝5分、と覚えておくと、工数データを読むときに迷いません。

作業内容 / time_work（さぎょうないよう：288枠に詰めた、その日の作業の記録） 上記288枠の中身を表す文字列フィールド。1枠ずつに「どの工数区分か」を表す記号が入り、1日分でつながって1つの長い文字列になります。**たとえ:** 288マスの塗り絵を、1列の暗号文字に直したもの。1文字（または1区切り）が1つの5分マスに対応。

作業詳細 / detail_work（さぎょうしょうさい：作業の補足説明） `detail_work`（ `max_length=32676` ）。288枠だけでは書ききれない、作業の詳しい説明を入れる、とても長い文字列の欄。**たとえ:** 塗り絵の余白に書く「メモ欄」。色だけでは伝わらない事情を文章で補足する。

工数区分定義 / kosu_division（こうすうくぶんていぎ：作業の種類の一覧の定義） どんな作業の種類（区分）があるかを定義するモデル。「1バージョンにつき最大50区分」まで定義できます。`Business_Time_graph` の `def_ver2`（工数区分定義Ver）で、その記録がどのバージョンの定義に基づくかを示します。**たとえ:** 塗り絵の「色見本（パレット）」の定義。「赤＝組立、青＝検査…」と、どの色が何を意味するかを決めた一覧。

def_ver / 定義バージョン（ていぎばーじょん：使っている工数区分定義の版） 工数区分の定義は時々見直されます。「いつの定義に基づく記録か」を区別するためのバージョン名。過去の記録は当時のバージョンの意味で読む必

要があるため、記録ごとにバージョンを保存しています。**たとえ:**「○年度版の色見本」。年度で色の意味が変わることがあるので、どの年度の見本で塗ったかを記録しておく。

就業日 / work_day（しゅうぎょうび：その工数記録が、どの日のものか） `work_day2`（就業日）。その作業記録がいつの日のものかを表す日付。検索や集計の基準になります。**たとえ:** 日記の「日付」。「いつの記録か」を示す。

休憩文字列（きゅうけいもじれつ：休憩時間を表す独自の文字列） 従業員モデルの `break_time1` ～（`max_length=9`）など、**休憩の時間帯を文字列で表したデータ**。直ごと・残業ごとに複数持ちます。工数の集計時、この休憩文字列を解釈して「実働時間から休憩を差し引く」処理が行われます。最近のコミット `休憩変更時の工数消去バグ修正` は、この休憩文字列の扱いに関わる修正です。**たとえ:** タイムカードの「休憩の時間メモ」。「12:00-12:45」のような文字を、後で機械が読み取って実働を計算する。

> ⚠️ **休憩文字列は要注意ポイント。**工数のバグは、この休憩文字列の解釈まわりで起きやすいです（履歴のバグ修正コミットがそれを物語っています）。休憩に関わる修正をするときは、直ごと・残業ごとに欄が分かれていること、文字列の形式を壊さないことに特に注意してください。

残業時間 / over_time（ざんぎょうじかん：その日の残業の量） `over_time`（残業時間）。通常勤務を超えた分の時間。休憩文字列にも「残業休憩」が直ごとに用意されています。**たとえ:** 定時を過ぎた延長分のメーター。

Business_Time_graph（びじねすたいむぐらふ：1日分の工数記録を表すモデル）「誰が・いつ・どの直で・どの作業を（288枠）・残業や休憩はどうだったか」を1件にまとめた、工数記録の中心モデル。このアプリのデータの主役です。**たとえ:** 1人分・1日分の「作業日報」。これが何万枚も積み重なって、工数データベースになる。

team_member（ちーむめんばー：チーム（班）の構成を表すモデル） チーム（班）の編成を表すモデル。「1チーム最大15人まで」という上限があります。集計をチーム単位で行うために使われます。**たとえ:** 班のメンバー表。最大15人までの名簿。

権限 / authority・管理者 / administrator（けんげん・かんりしゃ：その人ができる操作の範囲） 従業員モデルの `authority`（権限）と `administrator`（管理者）。`true` / `false` で、編集できる範囲や管理機能を使えるかどうかを分けます。**たとえ:** 鍵の種類。一般の鍵（一般従業員）と、マスターキー（管理者）で開けられる部屋が違う。

AsyncTask / History（えいしんくたすく／ひすとりー：非同期処理の進捗と、変更の履歴を記録するモデル） `AsyncTask` はバックアップ等の裏処理の進み具合を、`History` はデータの変更履歴（監査ログ）を記録するモデル。`signals.py` と連携して、いつ何が変わったかを残します。**たとえ:** AsyncTask＝宅配の「追跡番号の状況」、History＝台帳の「訂正の履歴印」。

ポップアップ / pop_up（ぽっぷあっぷ：従業員ごとに表示する通知メッセージ） 従業員モデルの `pop_up1` ～ `pop_up5` と `pop_up_id1` ～。特定の従業員に対して画面に出す通知メッセージの欄。最近のコミット `通知誤削除修正` はこの通知まわりの修正です。**たとえ:** 個人宛の付箋（ふせん）メモ。その人がログインしたときに「お知らせ」として画面に貼り出す。

break_check（ぶれいくちぇっく：休憩エラーのチェックを有効にするかどうかのフラグ） 従業員モデルの `break_check`。休憩時間の入力に矛盾があるときにエラーを出すかどうかを、その人ごとに切り替えるためのスイッチ（`true` / `false`）。**たとえ:**「休憩のつじつまチェックを、この人にはする／しない」を切り替える設定スイッチ。

10. テストに関わる言葉

「コードがちゃんと動くか」を自動で確かめる **テスト** に関わる言葉です。このアプリのフロントは **Vitest（ヴィテスト）** でテストします。

> **テスト（てすと：コードが正しく動くかを確かめる作業／そのためのコード）** 「この関数に5を渡したら、ちゃんと10が返るか」のように、コードの正しさを自動で確認する仕組み・コード。修正後に「壊していないか」を素早く確かめられます。このアプリのテストは `frontend/src/tests/` にあります。**たとえ:** 出荷前の検品。「ボタンを押したら本当に動くか」を1つずつ確認する検査ライン。
>
> **Vitest（ヴィテスト：このアプリのフロントで使うテスト道具）** フロントエンドのテストを実行する道具。`npm test`（変更を見張りながら実行）、`npm run test:run`（1回だけ実行）、`npm run test:ui`（画面付きで実行）で動かします。**たとえ:** 検品ラインの「検査装置」。テストコードを流し込むと、合否を判定してくれる。
>
> **テストケース（てすとけーす：1つ1つの検査項目）** 「○○という入力のとき、△△になるはず」という、1件の検査。`it("...", () => { ... })` の形で書きます。**たとえ:** 検品リストの1項目。「赤いボタンを押す→ランプが光る」のような、確認すべき1ケース。
>
> **アサーション（あさーしょん：「こうなっているはず」と表明して確かめる文）** テストの中で「結果はこうであるべき」と宣言し、その通りかを判定する文。`expect(結果).toBe(期待値)` の形がよく使われ、`expect(...)` =「〜であることを期待する」、`.toBe(...)` =「〜と等しい」です。**たとえ:** 検品で「重さは100gのはず」と基準を表明して、はかりに乗せて確かめる行為。
>
> **モック（もっく：本物の代わりに使う「偽物の部品」）** テストのとき、本物のサーバー通信などを呼ぶと遅い・不安定なので、代わりに「決まった答えを返す偽物」に差し替えること。このアプリでは Axios（通信）や子コンポーネントをモックします。**たとえ:** 撮影用の「ダミーの料理（食品サンプル）」。本物を作る手間をかけず、見た目だけ本物そっくりの代用品で試す。
>
> **モックする（もっくする：本物を偽物に差し替える操作）** 「Axiosをモックする」=「通信部分を、決まった結果を返す偽物に置き換える」。これで通信の成否に左右されず、画面側のロジックだけを確実にテストできます。**たとえ:** 火災訓練で「本物の火」ではなく「煙の出る装置（ダミー）」を使う。本物の危険なし安全に、手順だけ確かめる。
>
> **React Testing Library（りあくと・てすていんぐ・らいぶらり：Reactの画面をテストする道具）** Vitestと一緒に使い、「この画面にこの文字が表示されているか」「ボタンを押したらどうなるか」を、利用者の目線で確認するためのライブラリ。**たとえ:** 「お客さん役」の検査員。実際に画面を見て、ボタンを押して、表示を確かめる。

11. 運用・Azure の言葉

作ったアプリを「本番で動かし続ける」運用に関わる言葉です。このアプリは **Microsoft Azure（マイクロソフト・アジュール）** という、クラウド上で動いています。

Azure（アジュール：Microsoftのクラウドサービス） インターネット越しに、サーバーやデータベースを借りられるMicrosoftのサービス。このアプリの本番環境はAzure上にあります。自前で機械を買わず、必要な分だけ借りて使えます。**たとえ:** レンタルオフィス。自分で建物を建てず、必要な広さの部屋を借りて事業をする。

クラウド（くらうど：インターネット越しに借りるコンピュータ資源） 自前の機械を持たず、インターネット越しに必要なだけサーバーやストレージを借りて使う方式。Azure、AWS、Google Cloudなどが代表。**たとえ:** 電気の購入。自宅に発電機を持たず、必要な分だけ電力会社から買う。

デプロイ（でぷろい：作ったアプリを本番環境に配置して動かすこと） 開発したコードを本番のサーバーに送り込み、利用者が使える状態にすること。「リリース」「公開」とも近い。**たとえ:** 完成した商品を「店頭に並べる」こと。倉庫（開発）から売り場（本番）へ出す。

フレキシブルサーバー（ふれきしぶるさーばー：Azureの柔軟なデータベース提供形態） Azureが提供するデータベース（PostgreSQLなど）のサービス形態の1つ。「Azure Database for PostgreSQL フレキシブルサーバー」のように呼ばれ、性能やバックアップ設定を柔軟に調整できるのが特徴。このアプリのデータベースはこの形態で動いていることがあります。**たとえ:** 広さや設備を後から調整できる「フレキシブルな貸し倉庫」。荷物（データ）が増えたら、棚を増やしたり管理人を増やしたりできる。

App Service（あっぷさーびす：Azureでアプリを動かす場所） Azure上でWebアプリ（このアプリのDjango部分など）を動かすためのサービス。サーバーの細かい管理をAzureに任せて、アプリを動かすことに集中できます。**たとえ:** 内装・電気・水道つきの「テナント区画」。建物の管理は大家（Azure）に任せ、店（アプリ）の運営に集中できる。

環境変数（かんきょうへんすう：本番の設定値をコードの外から与える） （section 3でも触れましたが運用で特に重要）本番のデータベース接続先やパスワードなどは、コードに書かず、Azureの設定（環境変数）として与えます。`DEBUG` を本番では `False` にする、なども環境変数で切り替えます。**たとえ:** 店ごとに違う「金庫の暗証番号」。同じ店舗マニュアル（コード）でも、暗証番号（設定）は店ごとに外から与える。

DEBUG（でばっぐ：開発用の詳細表示モードのオン／オフ） Djangoの設定。`True` だとエラー時に詳しい内部情報を表示します（開発に便利）。本番では情報漏れ防止のため必ず `False` にします。**たとえ:** 機械の「整備モード」。整備中（開発）は内部を全部見せるが、営業中（本番）はカバーを閉じて隠す。

ALLOWED_HOSTS（あらうど・ほすつ：アクセスを許可するドメインのリスト） Djangoの設定で、「このアドレスから来た要求だけ受け付ける」と許可するドメイン名のリスト。本番のドメインをここに登録します。**たとえ:** 「この住所宛の郵便だけ受け取る」という受信許可リスト。

バックアップ（ばっくあっぷ：データの控えを取っておくこと） 万一データが壊れたり消えたりしたときに備え、定期的にデータの控え（コピー）を保存しておくこと。このアプリは Django-Q の裏処理でバックアップ・復元を行います。**たとえ:** 大事な書類のコピーを別の場所に保管しておくこと。火事（障害）でも控えがあれば復元できる。

リストア／復元（りすとあ／ふくげん：バックアップから元に戻すこと） 取っておいたバックアップを使って、データを以前の状態に戻すこと。**たとえ:** コピーを使って、失われた書類を作り直すこと。

ダウンタイム（だうんたいむ：サービスが使えなくなっている時間） アプリが止まっていて利用者が使えない時間。運用では、これをできるだけ短くすることが大切です。**たとえ:** 店の「臨時休業」の時間。短いほどお客さんに迷惑がかからない。

12. 記号・略語のはやわかり表

コードでよく見る記号や略語を、まとめて引けるようにしました。「これ何の記号？」と思ったらここを。

記号・略語	読み	意味・使われ方
`//`	スラッシュ2つ	JS/TSの1行コメント（メモ。実行されない）
`/* */`	スラッシュアスタリスク	JS/TS/CSSの複数行コメント
`#`	シャープ／ハッシュ	Pythonの1行コメント／CSSではID指定
`{ }`	波かっこ／中かっこ	処理のまとまり／オブジェクト／JSXでの値の埋め込み
`[ ]`	角かっこ	配列／要素の番号アクセス
`( )`	丸かっこ	関数の引数／計算の優先
`=>`	アロー（矢印）	アロー関数（関数の短い書き方）
`===`	イコール3つ	厳密に等しいか（型も含めて比較。JS/TS）
`==`	イコール2つ	等しいか（型を緩く見る。JSでは `===` 推奨）
`&&`	アンド2つ	かつ／JSXでは「左がtrueなら右を表示」
`||`	パイプ2つ	または
`!`	エクスクラメーション	否定（〜でない）
`?`	クエスチョン	任意（あってもなくても）／三項演算子
`=`	イコール	代入（右の値を左に入れる）
`@`	アット	Pythonのデコレータ（`@login_required` など）
`.`	ドット	持ち物へのアクセス（`member.objects` など）
`:`	コロン	「名前: 値」／TSの型注釈／CSSのプロパティ
`;`	セミコロン	文の終わり（JS/TS/CSS）
`$`	ドル	テンプレート文字列の `${ }`（値の埋め込み）
`...`	ドット3つ	スプレッド（中身を展開する）／残余
API	エーピーアイ	プログラム同士の窓口（section 2）
DB	ディービー	データベース
DOM	ドム	HTMLの木構造（section 5）
DRF	ディーアールエフ	Django REST Framework
SPA	エスピーエー	シングルページアプリ（section 7）
JSX	ジェイエスエックス	JSにHTMLを混ぜる記法（section 5・7）

記号・略語	読み	意味・使われ方
ORM	オーアールエム	SQLを書かずDB操作（section 8）
CSRF	シーエスアールエフ	なりすまし送信防止（section 8）
CORS	コルス	別ドメインアクセス制御（section 8）
WSGI	ダブリュエスジーアイ	Pythonアプリとサーバーのつなぎ目（section 8）
PR	ピーアール	プルリクエスト（section 4）
TS	ティーエス	TypeScript
JS	ジェイエス	JavaScript
401	よんまるいち	「未ログイン」を表すHTTPエラー番号（section 2）

---

13. 困ったときの調べ方

用語集に載っていない言葉やエラーに出会ったとき、自力で調べる方法です。「自分で調べられる」ことが、引き継ぎ後にいちばん効いてきます。

13.1 エラーメッセージは「英語の手紙」だと思って読む

エラーメッセージは怖く見えますが、たいてい「どこで・何が・なぜ」を教えてくれる親切な手紙です。

> ▼ **調べ方の手順**: 1. **エラーの最後の行（または最初の行）を読む。**ここに要点があることが多い。2. **ファイル名と行番号を探す。** `Login.tsx:42` のような表記は「Login.tsxの42行目」。そこを開く。3. **エラーメッセージをそのまま検索エンジンに貼る。**自分の固有名（ファイル名や変数名）は外して、共通部分だけで検索すると当たりやすい。
>
> ▼ **こう読めれば成功**: `Cannot read properties of undefined (reading 'name')` →「undefined（中身がない）ものから `name` を読もうとして失敗した」→「nameを持っているはずの値が、実は空っぽだった」→その値がどこで作られるかをさかのぼる。

13.2 このアプリの中を検索する

「この言葉、このアプリのどこで使われている？」を調べるには、エディタの全文検索（VS Codeなら `Ctrl + Shift + F`）が最強です。

- 知らないクラス名・関数名・モデル名を検索すると、定義場所と使用場所が一覧で出ます。
- たとえば `Business_Time_graph` を検索すれば、それを使っている全ビューが見つかります。

> **保守の黄金フロー（再掲・全分野共通）**：「直したい部分を特定 → その名前で全文検索 → 定義と使用箇所を読む → 影響範囲を理解してから直す」。いきなり書き換えず、まず読む。これが壊さないコツです。

13.3 公式ドキュメントと信頼できる情報源

困ったら、まず **公式の説明** を見る習慣をつけましょう。検索結果の上位でも、古い情報や間違いがあります。

分野	調べ先（キーワード）
React	「React 公式 ドキュメント」「useState」「useEffect」
Django	「Django ドキュメント」「Django QuerySet」「Django REST Framework」
TypeScript	「TypeScript ハンドブック」
CSS	「MDN CSS （調べたいプロパティ名）」
Git	「Git 使い方」「git コンフリクト 解決」

> **用語: MDN（エムディーエヌ）** … Mozilla Developer Network。HTML・CSS・JavaScriptの、信頼できる定番の解説サイト。「困ったらMDN」が合言葉。

13.4 それでも分からないときの聞き方

人に聞くときは、次の3点を添えると、答えがもらいやすくなります。

1. **やりたいこと**（例：工数一覧で休憩を差し引いて表示したい）
2. **やったこと・試したこと**（例：`break_time1` を読んで差し引いてみた）
3. **起きたこと**（例：このエラーが出た／結果がこうなった、とエラー文や画面を添える）

> ⚠️ **「動きません」だけでは伝わりません。**「何を・どうしたら・どうなったか」をセットで伝えると、相手も自分も原因に早くたどり着けます。これは人に聞くときだけでなく、自分でデバッグするときの考え方そのものでもあります。

13.5 たとえ話を「自分用」に作る

新しい用語に出会ったら、この章のように **自分なりのたとえ話** を1つ作ってみてください。たとえ話を作れた言葉は、もう自分のものです。ノートやこのファイルの近くにメモしていくと、あなただけの用語集が育ちます。

---

14. この章のまとめ

- この章は **辞書**。通読不要。困ったら目次か `Ctrl + F` で引く。たとえ話は「最初のイメージの入口」であり、慣れたら本当の意味へ進む。
- **アプリ全体**：フロントエンド（画面・React）とバックエンド（裏側・Django）が、**API** を通じて **リクエスト／レスポンス**（JSON）をやりとりする。データは **PostgreSQL** に保管。未ログインは **401** で `/login` に戻る。
- **道具**：エディタ・ターミナル・npm（フロント）／pip（バック）・ビルド・`.env`（秘密設定）・ローカル／本番。
- **Git**：コミット（記録）→ブランチ（作業場）→プッシュ／プル（共有）→マージ（合流）。ぶつかったら **コンフリクト** を手で解決。提案は **PR**。
- **HTML/CSS**：**DOM**（操作できる木）、**ボックスモデル**（margin/padding/border/content）、**box-sizing**、**Flexbox**、**擬似要素**、**CSS変数**、**clamp/calc**。
- **JS/TS**：変数・関数・**型**・**非同期**（**Promise**・**async/await**）・イベント。
- **React**：**コンポーネント**を**props**（親→子）と**state**（移ろう状態・**useState**）で組み立て、**useEffect** で副作用を実行。**SPA** はページを丸ごと読み込み直さない。
- **Python/Django**：**モデル**（表の設計図）を **ORM**（**QuerySet**）で操作し、**マイグレーション**でDBに反映。**シリアライザ**でJSON変換、**ビュー**で処理。**セッション**で認証、**CSRF/CORS**で守り、**デコレータ/シグナル/ページネーション**を活用。本番は **WSGI**＋**Gunicorn**。
- **このアプリ特有**：**工数・直（ちょく）**・**288枠**（1日を5分刻み＝1440÷5）・**休憩文字列**（バグの要注意点）・**ショップ・工数区分定義**。中心モデルは `member` と `Business_Time_graph`。
- **テスト**：**Vitest**＋React Testing Library。**アサーション**（expect）で表明し、**モック**で本物を偽物に差し替える。
- **運用**：**Azure** に **デプロイ**。データベースは **フレキシブルサーバー**。設定は **環境変数**（本番は `DEBUG=False`）。
- **困ったら**：エラーは英語の手紙として要点を読む → 全文検索で定義・使用箇所をたどる → 公式ドキュメント（React/Django/MDN）→ 聞くときは「やりたいこと・試したこと・起きたこと」をセットで。

これで本資料の用語編は終わりです。全体の地図と読み進め方は、「README.md」に戻って確認してください。わからない言葉に出会ったら、いつでもこの章へ。あなたの引き継ぎが、少しでも安心して進みますように。